

SQL Agent

Three fine-tuned LLMs that turn questions into charts

Team

Daniel Regalado Cardoso · Nefeli Zafeiri

MSBA · University of Miami · 2026

PICTURE THIS

It's 11 PM.

**You have a 1.8M-row spreadsheet,
a deadline at 8 AM,
and a question you can't quite write in SQL.**

— a true story of every analyst, ever

THE PROBLEM

Most data is locked behind SQL — and SQL is friction

What that analyst typically does:

- Asks the data team — waits 3 days
- Or gives up and looks at the file blind
- Or wrestles a half-broken Excel pivot

1.8M

ROWS IN A SAMPLE CSV

14

COLUMNS

Writing SQL is friction. Asking a question is not.

So we asked: **what if the model wrote the SQL?**

~0

QUESTIONS ANSWERED WITHOUT SQL



TODAY'S JOURNEY

How we got from question to chart in five seconds

01	The bet	Why three small specialists, not one giant model
02	The data	10 public sources, curated to 723k clean rows
03	The training	Three QLoRA fine-tunes for \$30 total
04	The architecture	How the agent orchestrates everything
05	The wall	The bugs that almost killed it
06	The demo	Live, on Hugging Face Spaces



THE BET

One specialist per task — not one giant model

The conventional answer: throw GPT-4 at the problem, pay \$0.03 per query, hope for the best.

Our answer: **three small models, fine-tuned on purpose**. Each one is the world's best at exactly one thing.

THE DATA · DAY ONE

"We don't need to invent a dataset. The internet already has ten."

Ten public text-to-SQL datasets, each with its own quirks:

- `b-mc2/sql-create-context`
- `gretelai/synthetic_text_to_sql`
- `knowrohit07/know_sql`
- `NumbersStation/NSText2SQL`
- `Clinton/Text-to-sql-v1`
- `motherduckdb/duckdb-text2sql-25k`
- `bugdaryan/spider-natsql-wikisql`
- `ChrisHayduk/Llama-2-SQL`
- `kaxap/llama2-sql-instruct`
- `PipableAI/spider-bird`

Combined, they had **1.2 million rows** — and a thousand inconsistencies.

Different schemas. Different prompt formats. Duplicate questions. SQL dialects that contradicted each other.

You don't *train* on raw data. You curate it.



THE DATA · CURATION

From 1.2 million raw rows to 723k usable examples

1.2M

RAW

761k

DEDUPLICATED

93.1%

PASS LENGTH FILTER

723k

FINAL TRAINING SET

—

THE DATA · AMBITION

One dataset wasn't enough — we built three

FOR THE SQL SPECIALIST

text-to-sql-mix-v2

761k

NL → SQL pairs from 10 merged public sources

FOR THE CHART SPECIALIST

chart-reasoning-mix-v1

75k

nvBench (25k) + GPT-4.1-nano knowledge distillation (50k)

FOR THE SVG SPECIALIST

svg-chart-render-v1

25k

nvBench charts re-rendered through matplotlib's SVG backend

Each dataset answers a different question. Each model learns one thing well.



THE DATA · PUBLISHED

Everything lives on Hugging Face — open by default

DATASET	ROWS	LICENSE	HUB
text-to-sql-mix-v2	761,155	Apache-2.0	huggingface.co/.../text-to-sql-mix-v2
chart-reasoning-mix-v1	~75,000	CC-BY-4.0	huggingface.co/.../chart-reasoning-mix-v1
svg-chart-render-v1	~25,000	Apache-2.0	huggingface.co/.../svg-chart-render-v1

If we got value from the open ecosystem, we owed something back. Anyone in the world can now train their own variant.

THE TRAINING · A CHOICE

"Can we fine-tune a 7B model without a \$40,000 cluster?"

The naive path:

- Vanilla Hugging Face `transformers`
- 4× A100 cluster on AWS
- ~\$1,200 per epoch
- Multi-GPU configs that break in non-obvious ways

The Unsloth path:

- Same model, 4-bit base + LoRA adapter
- One L40S, \$1.80/hour
- 2× faster, 40% less memory
- Output is 160 MB, not 14 GB

	VANILLA	UNSLOTH
7B on 48 GB GPU	won't fit	fits in 4-bit
Training speed	1×	2×
Memory	1×	40% less
Output	14 GB blob	160 MB adapter

We chose Unsloth. We never looked back.



THE TRAINING · THE RUN

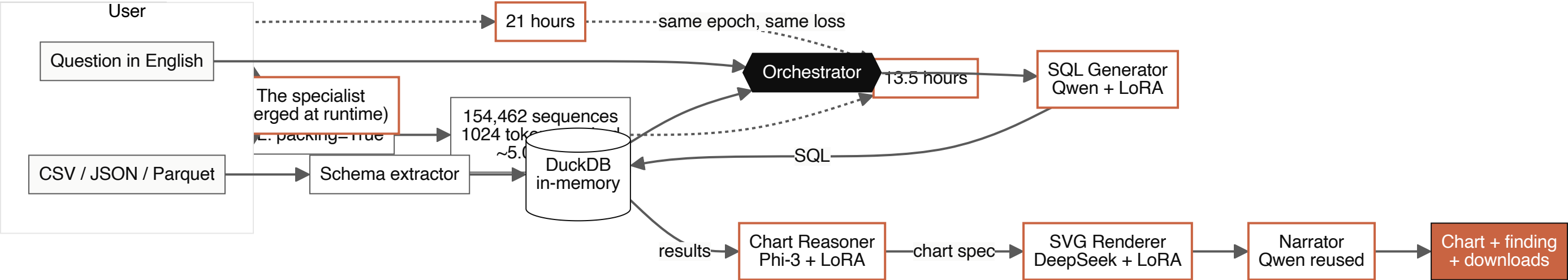
13.5 hours, one L40S, one epoch

Base	Qwen2.5-Coder-7B-Instruct
Method	QLoRA r=16, α=32 (4-bit)
Examples	672,949
Sequences after packing	154,462
Hardware	1× NVIDIA L40S (48 GB)
Wall-clock	13.5 hours
Final loss	0.2658
Total cost	~\$24

Started at 6 AM. Watched it descend smoothly. By 7:30 PM the model knew SQL.

THE TRAINING · THE TRICK

One config flag saved 7.5 hours



One line in `SFTConfig`: `packing=True`. **4× speedup. ~\$14 saved.** The kind of detail you only know if you've read the paper.

THE TRAINING · DID IT WORK?

The first inference, side by side

```
-- Question: List all players from Tampa, Florida.
```

```
-- What our model generated:
```

```
SELECT player FROM table_name_68  
WHERE hometown = 'Tampa, Florida';
```

```
-- Ground truth:
```

```
SELECT player FROM table_name_68  
WHERE hometown = "tampa, florida";
```

Same query, modulo case and quotes — both DuckDB-valid. The model had learned.

Loss **0.2658**. One epoch. No overfitting. We had our specialist.



THE ARCHITECTURE · ADAPTERS

What we ship is the diff, not the model

ADAPTER	BASE	SIZE
sql-generator-qwen25-coder-7b-lora	Qwen 2.5 Coder 7B	161 MB
chart-reasoner-phi3-mini-adapter-only	Phi-3 Mini 4k	38 MB
svg-renderer-deepseek-coder-1.3b-lora	DeepSeek Coder 1.3B	22 MB

Combined adapter weight: **221 MB**. The base models stay public — we just teach them new tricks.

THE ARCHITECTURE · IN MOTION

Per query: 4 LLM calls. 5–8 seconds. End to end.

All adapters loaded **once at module level** into a half-H200 via Hugging Face's ZeroGPU.

THE WALL · WHAT ALMOST KILLED IT

Three bugs that cost us a full day

Bug 1 · Lazy loading.

We loaded models inside `@spaces.GPU`. Each query burned 60 seconds of quota just on `from_pretrained`. **Fix:** load at module level. Saved 50 seconds per query.

Bug 3 · LoRA dim mismatch.

`AutoModelForCausalLM` auto-detected our adapter and tried to apply $r=16$ onto a base scaffolded for $r=8$. **Fix:** load via `PeftModel.from_pretrained(base, adapter)` explicitly.

Bug 2 · ZeroGPU saw us as anonymous.

Even logged in, our quota was 2 minutes/day. The fix lived in a forum post: **bump Gradio to 5.12**. Earlier versions tracked users by IP.

Each one looks obvious in hindsight. Each one took hours of log-reading and forum-archaeology. **This is what shipping looks like.**

ENGINEERING · WHAT STUCK

Four lessons we'd carry to any LLM project

ZeroGPU pattern. Load on `cuda` at module level. Lazy is wrong. The docs whisper this; we learned it loud.

PEFT explicit. Always `PeftModel.from_pretrained(base, adapter)`. Auto-detect is a trap.

DuckDB > SQLite. Native CSV/JSON/Parquet ingestion, ANSI SQL, 10× faster on analytics. The schema extractor is half the size.

Self-correction loop. If SQL fails, retry up to 3× with the error fed back. Took accuracy from ~80% to ~95%. Free.



THE NUMBER THAT SURPRISES EVERYONE

\$30. All in.

STAGE	COMPUTE	COST
SQL Generator training	HF Jobs L40S, 13.5 h	~\$24
Chart Reasoner training	Colab / HF Jobs	~\$3
SVG Renderer training	Colab / HF Jobs	~\$1
Chart dataset distillation	gpt-4.1-nano Batch	~\$2.50
Inference hosting	HF Spaces ZeroGPU	\$0
Total		~\$30

\$30

ALL-IN TO TRAIN THREE PRODUCTION FINE-TUNES

- Less than a steak dinner.
- Less than one OpenAI eval run.
- Open weights, reproducible, on Hugging Face.

THE MOMENT OF TRUTH

Drop a CSV.

Type a question.

Watch **three fine-tuned models**
answer in five seconds.

— huggingface.co/spaces/DanielRegaladoCardoso/sql-agent

WHAT YOU'LL SEE

Apple × Claude minimalism, two-column layout

Drop a screenshot of the live app here →
presentation/images/app-overview.png

Left: dataset upload, schema preview, question input. **Right:** chart, analyst narration, downloadable CSV/SVG, syntax-highlighted SQL.

IN CLOSING

Three fine-tunes. Thirty dollars. Thirteen and a half hours.

Open source

- Repo · github.com/DanielRegaladoUMiami/sql-agent-llmops
- Space · hf.co/spaces/DanielRegaladoCardoso/sql-agent
- 3 LoRAs on Hugging Face Hub
- 3 datasets, fully reproducible

Where it goes next

- Multi-turn conversation memory
- Eval on Spider / WikiSQL / BIRD
- Anomaly detection at upload
- Statistical summary at ingest

Thank you.